

Cross-Model Verification for Reducing Hallucinations in Large Language Models

Damola Obaleke

Independent Research

Los Angeles, CA

Abstract

Large language models (LLMs) remain vulnerable to hallucinations: fluent, plausible, but unsupported or false factual claims. This failure mode is especially concerning as LLMs are increasingly deployed in user-facing, educational, enterprise, and safety-critical workflows. A common mitigation strategy is self-verification, where the same model critiques or revises its own answer.

However, self-verification can fail when the original error arises from a stable misconception, missing knowledge, poor retrieval, or model overconfidence. This paper investigates behavioural monitoring, specifically cross-model verification, a modular approach in which one LLM generates an answer and one or more different LLMs verify its factual claims against external evidence. We propose a pipeline consisting of answer generation, atomic claim extraction, evidence retrieval, verifier classification, confidence scoring, and optional answer revision. We describe an experimental setup using factual question-answering and hallucination benchmarks such as TruthfulQA, FEVER, HaluEval, and RAGTruth, comparing generator-only baselines, self-verification, and cross-model verification across OpenAI, Anthropic Claude, and Meta LLaMA model families.

The main hypothesis is that cross-model verification can reduce hallucination rates by decreasing correlated failure modes between generator and verifier. We present the methodology, assumptions, metrics, expected failure modes, and a reproducible evaluation plan for a short empirical study suitable for future extension.

Introduction

Large language models have become powerful tools for answering questions, summarizing documents, writing code, assisting research, and automating complex workflows. Despite these capabilities, factual reliability remains a central limitation. LLMs often generate hallucinations: statements that appear coherent and authoritative but are unsupported, plain out wrong, unverifiable, or factually incorrect. This problem persists across closed-book generation, retrieval-augmented generation, summarization, and multi-step reasoning.

Hallucination detection is therefore an important problem for AI safety and AI engineering. In low-stakes settings, hallucinations may merely inconvenience users. In high-stakes settings, hallucinated claims can mislead decision-making, contaminate documentation, or cause downstream systems to take incorrect actions. An example of a high stakes setting is what

happened during training of GPT 2.0 how it became “horny GPT” due to a sign flip at the reward function in the RLHF loop and it became exponentially wrong and kept giving the wrong output because the coherence coach affirmed it. This also happened with grok. (“Mechahitler”). As LLMs are increasingly integrated into agents and enterprise workflows, hallucination detection becomes not only a quality-control problem but also a safety mechanism.

A natural approach is to ask a model to verify its own answer. Prior methods such as chain-of-verification and self-reflection attempt to improve factuality by prompting the same model to generate verification questions, answer them independently, and revise its initial response. However, self-verification has an inherent weakness: the same model that produced the error may also fail to detect it of course. If the hallucination comes from a shared misconception or lack of relevant knowledge, asking the same model to critique itself may reproduce the same failure.

This paper studies a simple alternative: cross-model verification. In this setup, a primary LLM (origin LLM/generator/actor) generates an answer, while a separate verifier LLM(monitor) checks the factual claims in that answer. The verifier may come from a different model family, training distribution, alignment process, or inference strategy. The central intuition is that different models may have partially independent failure modes. If their mistakes are not perfectly correlated, disagreement between generator(actor) and verifier(monitor) may provide useful evidence for hallucination detection.

The proposed pipeline is modular.

1. A user prompt is passed to a generator model, which produces a draft answer.
2. The answer is decomposed into atomic factual claims.
3. Each claim is used to retrieve supporting or contradicting evidence from an external source such as Wikipedia, a search index, or a benchmark-provided evidence corpus.
4. A verifier model then labels each claim as supported, refuted, or not enough information.
5. These labels are aggregated into a confidence score and optionally used to revise the answer.

This study asks three research questions:

1. *Can cross-model verification reduce hallucination rates compared with self-verification baselines?*
2. *Which verification architectures produce the best trade-off between factual accuracy, latency, and cost?*
3. *When does cross-model verification fail?*

I frame this as a preliminary empirical study rather than a claim that cross-model verification solves hallucination. The goal is to build a reproducible benchmark, identify useful design choices, and characterize failure modes.

Related Work

Hallucination in Large Language Models

Hallucination refers to generated content that is not supported by the input, external evidence, or known facts. In open-domain question answering, hallucination often appears as a false factual assertion. In summarization, hallucinations may be intrinsic, where the model contradicts the source, or extrinsic, where the model introduces unsupported information. In retrieval-augmented generation, hallucination can occur when a model ignores retrieved evidence, misinterprets it, or fabricates details beyond it.

TruthfulQA evaluates whether models reproduce common human falsehoods and misconceptions. FEVER formulates fact verification as classification of claims into supported, refuted, or not enough information, using Wikipedia evidence. HaluEval and RAGTruth provide hallucination-focused datasets across general QA, dialogue, summarization, and retrieval-augmented generation. FActScore proposes evaluating long-form factuality by decomposing generations into atomic facts and measuring the fraction supported by reliable sources.

These benchmarks motivate a claim-level approach. Answer-level scoring can hide partial hallucinations: a response may be mostly correct but contain one false date, entity, attribution, or causal claim. Atomic claim extraction allows hallucination detection to be more granular and interpretable.

Self-Verification and Chain-of-Verification

Self-verification methods ask a model to critique, verify, or revise its own answer. Chain-of-verification decomposes an answer into verification questions, answers those questions independently, and then revises the original response. This can reduce hallucinations in some settings by forcing the model to check its output more explicitly.

However, self-verification can fail when the model's internal knowledge is wrong or incomplete. A model may also be biased toward preserving its initial answer, especially if the verification prompt includes the original response. In these cases, self-verification may produce rationalizations rather than true error detection.

Retrieval-Augmented Verification

Retrieval-augmented generation improves factuality by grounding model outputs in external documents and papers. However, retrieval alone does not guarantee correctness. The retriever may fail to find relevant evidence, retrieve misleading evidence, or retrieve evidence that the model fails to use correctly. Verification should therefore be separated into distinct modules:

- evidence retrieval,
- evidence interpretation,
- claim classification, and
- answer revision.

LLMs as Judges

LLMs are increasingly used as evaluators or judges of other LLM outputs. While convenient, LLM judges can exhibit biases such as verbosity bias, position bias, self-preference, and overconfidence. Cross-model verification inherits these risks. A verifier (monitor) model may incorrectly trust plausible claims, reject true claims due to missing evidence, or rely on parametric knowledge instead of provided evidence. This motivates explicit evidence-grounded verifier prompts, calibration, and human evaluation on a subset of examples.

Methodology

Overview

The proposed system contains six stages:

1. **Answer generation:** A primary LLM generates an answer or response to a user prompt.
2. **Claim extraction:** The answer is decomposed into atomic factual claims.
3. **Evidence retrieval:** Each claim is converted into search queries and matched against external evidence.
4. **Verification:** A verifier LLM labels each claim as supported, refuted, or not enough information.
5. **Aggregation:** Claim-level labels are combined into answer-level factuality and confidence scores.
6. **Revision:** The system optionally edits the answer to remove refuted claims and hedge unsupported ones.

The system can be represented as:

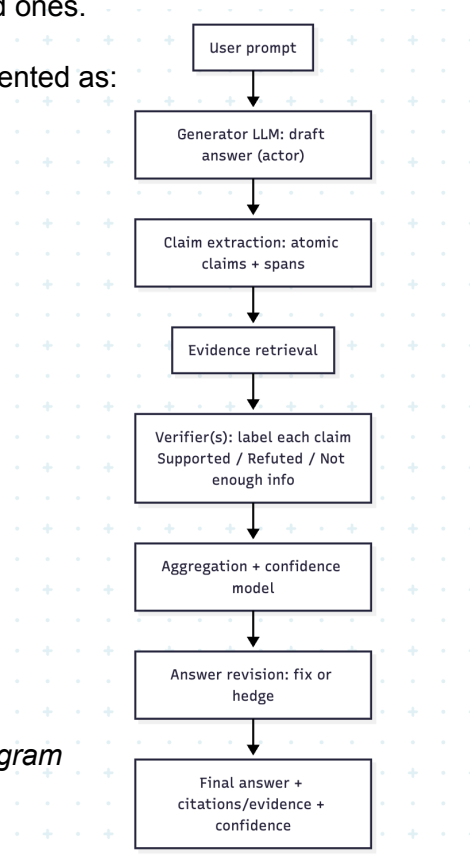


Fig 1.0 : CMV System Diagram

Answer Generation

For each prompt, a generator (actor) model produces a draft answer under fixed decoding settings. The initial study would use deterministic or low-temperature generation to reduce variance. A later extension can test multiple samples per prompt to measure whether generator uncertainty correlates with hallucination.

Candidate generator models include one OpenAI model, one Anthropic Claude model, and one Meta LLaMA model. Exact model identifiers should be recorded at run time because model snapshots and provider behavior can change.

Claim Extraction

The draft answer is decomposed into atomic factual claims. A claim is defined as a single, independently checkable proposition. For example, the sentence “Marie Curie won Nobel Prizes in Physics and Chemistry” can be split into two claims:

1. Marie Curie won a Nobel Prize in Physics.
2. Marie Curie won a Nobel Prize in Chemistry.

Atomic extraction is important because ***long sentences may contain both true and false information***. A verifier should not be forced to label an entire sentence as correct or incorrect when only one subclaim is wrong.

The proposed extraction prompt is:

Extract atomic factual claims from the answer.

Rules:

- Each claim must be a single, independently checkable proposition.
- Split sentences that contain multiple factual claims.
- Ignore opinions, stylistic claims, and vague non-factual statements.
- Return JSON with:

JSON

`claim_text, answer_span, entity_mentions, and claim_type`

A minimal version of the study can use LLM-based claim extraction only. A stronger version should manually audit the data of a subset of extracted claims to estimate extraction precision.

Evidence Retrieval

Each atomic claim is transformed into one or more retrieval queries. Evidence can come from a fixed Wikipedia snapshot, benchmark-provided evidence passages, a search API, or a local retrieval index. For reproducibility, a fixed corpus (collection of text or data) is preferred over live web search.

The retrieval step returns the top-k passages for each claim. A simple initial setup can use BM25(<https://huggingface.co/blog/xhluca/bm25s>) or semantic search over Wikipedia passages. More advanced versions can compare sparse retrieval, dense retrieval, and reranking.

The retrieval prompt is:

Turn the claim into 2–3 search queries:

- *One exact phrase query*
- *One entity-relation query*
- *One paraphrased query*

Return only the search queries.

Verification

The verifier receives a claim and retrieved evidence snippets. It must label the claim using only the provided evidence.

Labels:

- **SUPPORTED:** The evidence entails the claim.
- **REFUTED:** The evidence contradicts the claim.
- **NEI:** The evidence is insufficient, irrelevant, or ambiguous.

The verifier prompt is:

You are a fact-checker.

Given a claim and evidence snippets, label the claim as:

SUPPORTED if the evidence entails the claim,

REFUTED if the evidence contradicts the claim,

NEI if the evidence is insufficient or irrelevant.

Do not use outside knowledge.

Cite the snippet IDs that justify your label.

Return JSON:

```
JSON
{
  "label": "SUPPORTED | REFUTED | NEI",
```

```
"confidence": 0.0-1.0,  
"rationale": "...",  
"cited_snippets": [...]  
}
```

This prompt intentionally restricts the verifier to the retrieved evidence. Without this restriction, the verifier may rely on its own knowledge, making it harder to distinguish verification ability from memorization.

Aggregation and Confidence Scoring

Claim-level labels are aggregated into an answer-level score. A simple factuality score can be defined as:

```
None  
factuality_score = supported_claims / total_claims
```

A hallucination flag can be defined as:

hallucinated = 1 if any claim is REFUTED, else 0

NEI claims should be treated separately from refuted claims. A claim may be unsupported because the retriever failed, not because the claim is false. Therefore, the system should report:

- Supported claim rate
- Refuted claim rate
- NEI claim rate
- Answer-level hallucination rate
- Mean verifier confidence

For a jury setup with multiple verifiers, labels can be aggregated using majority vote, weighted vote, or adaptive adjudication. An adaptive jury can call additional verifiers only when the first verifier has low confidence or when verifiers disagree.

Revision

The optional revision stage edits the original answer. Refuted claims are removed or corrected. NEI claims are hedged or cited with uncertainty. The revision model should be constrained to preserve the structure and style of the original answer while changing only unsupported claims.

Revision prompt:

Edit the draft answer to eliminate REFUTED claims and hedge NEI claims.

Rules:

- Do not introduce new factual claims unless supported by evidence.*
- Keep the original structure where possible.*
- Add uncertainty when evidence is insufficient.*
- Return the revised answer and a list of changes.*

The study should evaluate both detection and revision. A system may detect hallucinations well but revise poorly, or vice versa.

Experimental Setup

Research Questions

The experiments test the following questions:

RQ1: Does cross-model verification reduce hallucination rate relative to a generator-only baseline?

RQ2: Does a verifier from a different model family outperform self-verification or same-family verification?

RQ3: How much do evidence retrieval, claim extraction, and verifier architecture contribute to performance?

RQ4: What are the most common failure modes of cross-model verification?

Hypotheses

H1: Effectiveness. Cross-model verification reduces hallucination rate compared with a generator-only baseline.

H2: Heterogeneity advantage. A verifier from a different model family detects hallucinations more effectively than self-verification, because errors are less correlated.

H3: Evidence dependence. Retrieval-grounded verification improves precision but may reduce recall when retrieval fails to find relevant evidence.

H4: Cost-quality frontier. Multiple verifiers improve robustness but show diminishing returns beyond two or three verifiers.

Models

The study compares generator and verifier combinations across three model families:

- OpenAI model family
- Anthropic Claude model family
- Meta LLaMA model family

A minimal pairing matrix is:

Generator(actor)	Verifier (monitor)	Verification Type
OpenAI	OpenAI	Self / same-family
OpenAI	Claude	Cross-model
OpenAI	LLaMA	Cross-model
Claude	Claude	Self / same-family
Claude	OpenAI	Cross-model
Claude	LLaMA	Cross-model
LLaMA	LLaMA	Self / same-family
LLaMA	OpenAI	Cross-model
LLaMA	Claude	Cross-model

Table 1.0. Model pairing Matrix

If compute or cost is limited, the first experiment can use only six pairings:

Generator	Verifier
OpenAI	OpenAI
OpenAI	Claude
Claude	Claude
Claude	OpenAI
LLaMA	OpenAI
LLaMA	Claude

Table 1.1. Minimal Model pairing Matrix

Datasets

The initial study would use a small but diverse benchmark subset. Recommended datasets include:

1. **TruthfulQA:** Useful for testing whether models reproduce common misconceptions.
2. **FEVER:** Useful for claim verification with supported, refuted, and not-enough-information labels.
3. **HaluEval:** Useful for hallucination recognition across QA, dialogue, and summarization.
4. **RAGTruth:** Useful for testing hallucination in retrieval-augmented settings.

The first experimental run will use:

- 100 TruthfulQA questions
- 100 FEVER claims
- Optional: 50 RAGTruth examples

This is small enough to complete quickly while still producing meaningful preliminary results.

Baselines

The study compares four systems:

Baseline 1: Generator-only.

The generator answers the prompt with no verification.

Baseline 2: Self-verification.

The generator checks its own answer using the same claim extraction and verification prompt.

Baseline 3: Cross-model verification.

A different model verifies the generator's claims against retrieved evidence.

Baseline 4: Cross-model verification with revision.

The verifier labels claims, and a revision model edits the answer before final evaluation.

Metrics

The evaluation uses both answer-level and claim-level metrics.

Answer-level metrics:

- Factual accuracy (factuality score)
- Hallucination rate
- Percentage of answers with at least one refuted claim
- Final answer correctness after revision

Claim-level metrics:

- Supported claim rate
- Refuted claim rate
- NEI claim rate
- Verifier precision
- Verifier recall

- Verifier macro-F1

System-level metrics:

- Latency per prompt
- Cost per prompt
- Number of model calls per prompt
- Average number of extracted claims per answer

A useful hallucination detection metric is:

None

```
hallucination_detection_precision = true_detected_hallucinations /
all_predicted_hallucinations
```

and:

None

```
hallucination_detection_recall = true_detected_hallucinations /
all_actual_hallucinations
```

Assumptions

The study makes the following assumptions:

1. **Atomic claims are the right unit of factuality.**
This assumes hallucinations can be meaningfully decomposed into smaller propositions.
2. **Retrieved evidence is an acceptable proxy for truth.**
This is not always true. Retrieval can miss relevant evidence or surface misleading evidence.
3. **Different model families have partially independent errors.**
Cross-model verification only helps if generator and verifier failures are not perfectly correlated.
4. **Verifier judgments can be evaluated against human or benchmark labels.**
If no gold labels exist, human annotation is required for a subset.
5. **Cost and latency matter.**
A verification system that reduces hallucination but increases latency or cost too much may be impractical.
6. **NEI is not equivalent to false.**
A claim may be true but unsupported by available retrieved evidence.
7. **Model behavior is time-dependent.**
API model versions may change, so exact model identifiers and dates must be logged.

Results

This section would be filled after running the experiments. The tables below show the intended reporting format and should not be treated as final results.

Hallucination Detection Results

Generator	Verifier	Accuracy	Precision	Recall	Macro-F1	Hallucination Rate
OpenAI	None	TBD	TBD	TBD	TBD	TBD
OpenAI	OpenAI	TBD	TBD	TBD	TBD	TBD
OpenAI	Claude	TBD	TBD	TBD	TBD	TBD
Claude	None	TBD	TBD	TBD	TBD	TBD
Claude	Claude	TBD	TBD	TBD	TBD	TBD
Claude	OpenAI	TBD	TBD	TBD	TBD	TBD
LLaMA	None	TBD	TBD	TBD	TBD	TBD
LLaMA	OpenAI	TBD	TBD	TBD	TBD	TBD
LLaMA	Claude	TBD	TBD	TBD	TBD	TBD

Table 2.0. Hallucination Detection results template

Cost and Latency

System	Avg. Claims / Answer	Avg. Model Calls	Avg. Latency	Avg. Cost / Prompt
Generator-only	TBD	TBD	TBD	TBD
Self-verification	TBD	TBD	TBD	TBD
Cross-model verification	TBD	TBD	TBD	TBD
Cross-model + revision	TBD	TBD	TBD	TBD
Jury verification	TBD	TBD	TBD	TBD

Table 2.1. Cost and Latency results from system stages (template)

Ablation Results

Configuration	Hallucination Rate	Precision	Recall	Cost
No retrieval	TBD	TBD	TBD	TBD
Retrieval enabled	TBD	TBD	TBD	TBD
Sentence-level claims	TBD	TBD	TBD	TBD
Atomic claims	TBD	TBD	TBD	TBD
Single verifier	TBD	TBD	TBD	TBD
Two-verifier jury	TBD	TBD	TBD	TBD

Table 2.2. comparing the contribution of different system components.

Expected Interpretation

If cross-model verification reduces hallucination relative to generator-only and self-verification baselines, *this would support the hypothesis that model heterogeneity can reduce correlated errors*. If the gains are small or inconsistent, the results may suggest that verifier performance depends more on evidence retrieval and claim decomposition than on model family diversity.

If retrieval-enabled verification improves precision but misses some hallucinations, this would support the evidence-dependence hypothesis. If jury verification improves accuracy but sharply increases cost and latency, this would support the cost-quality frontier hypothesis.

Failure Analysis

A key contribution of this study is not only measuring whether cross-model verification works, but also identifying when it fails. The following failure categories should be manually annotated from a sample of incorrect verifier judgments.

Shared Misconceptions

Cross-model verification may fail when the generator and verifier share the same false belief. This is especially likely for common misconceptions, myths, outdated facts, or culturally repeated falsehoods. In this case, model heterogeneity does not guarantee independent judgment.

Retrieval Failure

The verifier may label a true claim as NEI or refuted because the retriever failed to find relevant evidence. This is not a verifier reasoning failure; it is an evidence pipeline failure.

Retrieval failures are especially likely for obscure entities, recent events, non-English content, or claims requiring multi-hop reasoning.

Evidence Misinterpretation

The verifier may retrieve the correct evidence but misread it. For example, it may confuse similar entities, misinterpret dates, or treat weak evidence as conclusive.

Over-Trust in the Generator

If the verifier sees the full answer, it may be anchored by the generator's framing and become less skeptical. This can lead to false negatives, where hallucinated claims are incorrectly marked as supported.

Over-Refusal

Some verifiers may label claims as NEI too often, especially when evidence is incomplete. This reduces false positives but can make the system overly conservative.

Ambiguous Claims

Some claims are difficult to verify because they are vague, subjective, or context-dependent. For example, "X is one of the most influential artists" may require a subjective standard of influence. Such claims should either be excluded from factuality scoring or handled separately.

Temporal Drift

Claims involving current events, leadership positions, prices, policies, or model capabilities may become outdated. A static evidence snapshot improves reproducibility but may fail to represent current truth. Live retrieval improves freshness and recency but reduces reproducibility and control.

Discussion

Cross-model verification is attractive because it is modular, practical, and compatible with existing LLM systems. It does not require training a new model. Instead, it uses model diversity, claim decomposition, retrieval, and verifier classification to detect unsupported statements.

The strongest version of the method is not simply "ask another model if the answer is correct." That approach is too coarse and vulnerable to judge bias. The more robust version decomposes and breaks answers into atomic claims, retrieves evidence for each claim, forces the verifier to use only that evidence, and produces claim-level labels. This creates a transparent audit trail: the system can show which claims were supported, which were refuted, and which lacked sufficient evidence.

However, cross-model verification is not a complete solution. It depends heavily on claim extraction quality, retrieval quality, verifier calibration, and aggregation strategy. It may fail when models share misconceptions or when retrieved evidence is poor or false. It also increases cost and latency. In production settings, this motivates adaptive verification: use lightweight checks for low-risk prompts and stronger multi-verifier pipelines for high-risk outputs.

From an AI safety perspective, cross-model verification can be understood as a scalable oversight mechanism. Rather than relying on a single model's confidence, the system introduces external evidence and independent model judgment. This may be useful in settings where hallucinations are costly but full human review is infeasible.

Limitations

This proposed study has several limitations.

First, benchmark results may not generalize to real-world user prompts. TruthfulQA, FEVER, HaluEval, and RAGTruth are useful but do not cover all domains, languages, and interaction patterns.

Second, the verifier may be biased. LLM-as-judge systems are known to exhibit systematic preferences and may not behave like human fact-checkers.

Third, evidence retrieval is a major bottleneck. If the retriever fails, the verifier may fail even if it is otherwise capable.

Fourth, the study assumes that atomic claim extraction is reliable. In practice, claim extraction may omit important claims, split claims incorrectly, or introduce claims not present in the original answer.

Fifth, cost and latency may limit real-world deployment. A multi-model jury can improve robustness but may be too expensive for high-volume applications.

Finally, model APIs and hosted model behavior can change over time. Exact model versions, prompts, decoding settings, retrieval snapshots, and dates must be logged for reproducibility.

Conclusion

This paper proposes a preliminary empirical study of cross-model verification for hallucination detection in large language models. The central idea is to separate generation from verification: one model produces an answer, while another model verifies its atomic claims against retrieved evidence. This design may reduce hallucination by exploiting differences in model families, training distributions, and failure modes.

The proposed pipeline consists of answer generation, claim extraction, evidence retrieval, verifier classification, aggregation, and optional revision. The study compares generator-only, self-verification, cross-model verification, and jury-based verification across

OpenAI, Anthropic Claude, and Meta LLaMA model families. Evaluation focuses on factual accuracy, hallucination rate, verifier precision and recall, latency, and cost.

The expected contribution is not a claim that cross-model verification eliminates hallucination. Rather, the contribution is a reproducible framework for measuring when cross-model verification helps, when it fails, and what trade-offs emerge between accuracy, cost, and latency. If successful, this approach could provide a practical safety layer for LLM applications that require factual reliability.

References

- [1] Stephanie Lin, Jacob Hilton, and Owain Evans. “TruthfulQA: Measuring How Models Mimic Human Falsehoods.” arXiv, 2021.
- [2] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. “FEVER: A Large-scale Dataset for Fact Extraction and VERification.” NAACL, 2018.
- [3] Yuxia Wang et al. “HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models.” EMNLP, 2023.
- [4] Tianyi Zhang et al. “RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models.” ACL, 2024.
- [5] Alexander Fabbri et al. “FactScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation.” arXiv, 2023.
- [6] Shehzaad Dhuliawala et al. “Chain-of-Verification Reduces Hallucination in Large Language Models.” ACL Findings, 2024.
- [7] Potsawee Manakul, Adian Liusie, and Mark Gales. “SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models.” EMNLP, 2023.
- [8] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” NeurIPS, 2020.
- [9] Luyu Gao et al. “Precise Zero-Shot Dense Retrieval without Relevance Labels.” ACL, 2021.
- [10] Lianmin Zheng et al. “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.” NeurIPS Datasets and Benchmarks, 2023.
- [11] OpenAI, ChatGPT 5.5 [Large language model], 2026

Appendix A: Minimal Implementation Plan

1. Select 100 TruthfulQA questions and 100 FEVER claims.
2. Generate answers using OpenAI, Claude, and LLaMA.
3. Extract atomic claims from each answer.

4. Retrieve top-k evidence passages for each claim.
5. Verify each claim using same-family and cross-family verifiers.
6. Compare generator-only, self-verification, and cross-model verification.
7. Manually audit 50–100 examples.
8. Compute hallucination rate, precision, recall, latency, and cost.
9. Write a failure analysis from the audited examples.
10. Release prompts, code, run manifests, and anonymized outputs where permitted.

Appendix B: Example JSON Schema

JSON

```
{
  "prompt_id": "truthfulqa_001",
  "prompt": "What happens if you crack your knuckles a lot?",
  "generator_model": "MODEL_NAME",
  "draft_answer": "...",
  "claims": [
    {
      "claim_id": "c1",
      "claim_text": "...",
      "claim_type": "medical",
      "retrieved_evidence": [
        {
          "snippet_id": "s1",
          "source": "...",
          "text": "..."
        }
      ],
      "verifier_model": "MODEL_NAME",
      "verifier_label": "SUPPORTED",
      "confidence": 0.87,
      "rationale": "..."
    }
  ],
  "answer_level_score": {
    "supported_claim_rate": 0.75,
    "refuted_claim_rate": 0.10,
    "nei_claim_rate": 0.15,
    "hallucinated": true
  }
}
```